# Zero-Copy Continuous-Time Neural Networks for Bare-Metal Autonomous Racing on Microcontrollers

Manuel Yobani Martinez Sanchez

***Abstract*—Standard TinyML inference frameworks impose a dynamic SRAM tensor arena exceeding 40 KB for a 4,000-parameter network, consuming 8% of an ESP32-S3's addressable RAM before a single neural computation executes. Applying 8-bit post-training quantization (PTQ) to closed-form continuous-time (CfC) networks compounds this constraint with a second, previously undocumented failure: at 20 Hz control rates ($\Delta t = 0.05$ s), the time-gate pre-activation $f(\cdot)\Delta t$ rounds identically to zero under INT8 arithmetic, collapsing the temporal memory gate to a constant $t_{gate} = 0.5$ and degenerating the governing ODE into a time-invariant blend with no sequential memory. We formalize this failure as PTQ Collapse, confirm an 82% empirical fitness regression (22,500→4,100), and resolve both barriers with OmniTrain, an end-to-end framework for bare-metal deployment of continuous-time policies on commodity microcontrollers. OmniTrain embeds the INT8 grid directly into the mutation operator of a distributed Evolution Strategy (QAT-ES), producing quantized weights without a gradient-based Straight-Through Estimator. The resulting SparseCfC – a 75%-sparse, 270-synapse Neural Circuit Policy connectome – deploys through `.omnibit`, a Zero-Copy binary format that maps Compressed Sparse Row (CSR) weight blobs from SPI Flash to the Xtensa LX7 instruction cache via `mmap()`, reserving SRAM exclusively for the hidden state and allocating exactly 0 bytes to weights. In simulated F1TENTH racing, the QAT-evolved INT8 SparseCfC exceeds its Dense-FP32 predecessor by 132% (fitness 52,312 vs. 22,500) and drives 4.8 km without collision. On an ESP32-S3, OmniEngine executes one inference step in 1.22 ms using 1.5 KB of SRAM – a 96% reduction against TFLite Micro – while storing the full network in 14.2 KB of Flash, 4.5× smaller than the equivalent TFLite FlatBuffer.**

***Index Terms*—TinyML, Continuous-Time Neural Networks, Zero-Copy, Execute-in-Place, ESP32-S3, Autonomous Racing, Neural Circuit Policies, Quantization-Aware Training, Embedded Systems.**

## I. Introduction

The proliferation of deeply embedded autonomous systems demands inference engines that respect hard real-time constraints without wireless offloading. High-speed tasks such as F1TENTH autonomous racing [1] require agents to close control loops within 50 ms while processing 1D LiDAR scans on highly constrained platforms such as the ESP32-S3 (Xtensa LX7, 240 MHz, 512 KB SRAM). Physical sensor streams complicate this budget further: packet loss, variable I2C bus timing, and computational stalls introduce temporal irregularities that systematically degrade discrete-time recurrent networks such as LSTMs [2], whose gating assumes a fixed sampling interval.

Continuous-time, ODE-based networks address this assumption directly. Liquid time-constant (LTC) networks [3] define each neuron's hidden state through an ODE in which a learned, input-dependent term modulates an effective time-constant, and closed-form continuous-time (CfC) networks [4] provide an analytic approximation to that ODE's solution – replacing the iterative numerical solvers required by general Neural ODEs [7] with a single algebraic expression parameterized by the elapsed interval $\Delta t$. Because $\Delta t$ enters this expression explicitly, CfC networks inherit LTC's robustness to irregular sampling without paying the per-step cost of numerical integration. Neural Circuit Policies (NCPs) [5] complement this formulation with a wiring prior drawn from the *C. elegans* connectome, restricting sensory-, inter-, command-, and motor-neuron connectivity to a sparse, directed topology and reducing parameter count to biological proportions.

Two compounding barriers have nonetheless kept these networks off bare-metal microcontrollers. First, existing TinyML frameworks – TFLite Micro [6] chief among them – mandate a dynamic tensor arena sized to the full weight-plus-activation footprint, an allocation strategy inherited from server-class inference that consumes the majority of an MCU's addressable SRAM before a single computation executes. Second, and more fundamentally, applying standard 8-bit PTQ to a CfC's time-gate produces a collapse we characterize for the first time in this letter: at the sampling rates typical of robotic control, the gate's pre-activation is small enough relative to the INT8 quantization step that it rounds to exactly zero, silently reducing the network's temporal memory to a fixed 50/50 blend.

This letter makes three contributions:

1) **Formal PTQ Collapse analysis** (Section II): we prove INT8 precision is structurally insufficient to preserve a CfC time-gate at standard robotic sampling rates, and confirm an 82% empirical fitness regression.
2) **Evolutionary QAT, QAT-ES** (Section III): a distributed Evolution Strategy that snaps every candidate to the INT8 grid before fitness evaluation, replacing the gradient-based Straight-Through Estimator with a mechanism native to non-differentiable, reward-driven training, and producing INT8 policies that outperform their FP32 counterparts.
3) **The `.omnibit` Zero-Copy engine** (Section III): a compact CSR binary format for execute-in-place SPI Flash that reduces SRAM weight allocation to exactly 0 bytes, yielding a 96% SRAM and 85% latency reduction against TFLite Micro.

M. Y. Martinez Sanchez is an independent researcher (e-mail: contact@omnitrain.ai). Code, trained models, and datasets are openly available at https://github.com/mrmyms/Omnitrain to support reproducibility.

## II. Background

### A. Liquid Time-Constant Dynamics and the PTQ Collapse

The CfC closed-form hidden-state update [4] over a control step $\Delta t$ is

$$x(t+\Delta t) = \underbrace{\sigma(-f_\theta \Delta t)}_{t_{gate}} \odot \tanh(g_\theta) + [1 - t_{gate}] \odot \tanh(h_\theta) \tag{1}$$

where a shared backbone feeds three head networks: $f_\theta$ acts as the liquid time-constant driving the sigmoidal time-gate, while $g_\theta$ and $h_\theta$ construct the candidate and historical nonlinearities of the cell. $t_{gate}$ is the network's temporal arbiter, continuously modulating memory as a function of the elapsed time $\Delta t$ rather than a fixed step count.

**Proposition 1** (ODE Gate Collapse Under INT8 PTQ)**.** *Let the time-gate network $f(\cdot;\theta_f)$ be INT8-quantized with per-tensor scale $\Delta_q = \max|\theta_f|/127$. For any activation satisfying $|f(x,I)| \le \Delta_q/\Delta t$, the rounded pre-activation $\lfloor f_\theta \Delta t/\Delta_q \rceil = 0$, collapsing $t_{gate} = \sigma(0) = 0.5$ identically and degenerating (1) to a time-invariant blend with no sequential memory.*

*Proof.* Post-training quantization maps a real value $v$ to the integer grid via $Q(v) = \lfloor v/\Delta_q \rceil$, clamped to $[-127, 127]$, and reconstructs $\hat{v} = Q(v)\Delta_q$ before the nonlinearity is applied. The value entering the sigmoid in (1) is not $f_\theta$ in isolation but its product with the control interval, $v = f_\theta \Delta t$. Under the hypothesis $|f_\theta| \le \Delta_q/\Delta t$, we get $|v| \le \Delta_q$ and $|v/\Delta_q| \le 1$; since the pre-activation distribution concentrates well inside this bound, $Q(v) = 0$ for the majority of activations and $\hat{v} = 0$ regardless of the true, unquantized value of $f_\theta$. Consequently $t_{gate} = \sigma(0) = 0.5$ at every timestep, and (1) reduces to $x(t+\Delta t) = 0.5\tanh(g_\theta) + 0.5\tanh(h_\theta)$ – a fixed convex combination with no dependence on $\Delta t$, and hence no mechanism left to distinguish a densely sampled step from a sparsely sampled one. □

**Corollary 1** (20 Hz Collapse Threshold)**.** *At the F1TENTH control rate ($\Delta t = 0.05$ s) with an empirically calibrated $\Delta_q \approx 0.008$, the bound above evaluates to $|f_\theta| \le 0.16$, a range containing the majority of gate pre-activations near the weight distribution's mean.*

Fig. 1 visualizes the consequence: the PTQ gate collapses immediately, while the QAT-evolved model of Section III maintains structured temporal variation. This matches the empirical regression already summarized in Fig. 3 (fitness 22,500→4,100, −82%), confirming the corollary at operating conditions.

## III. The OmniTrain Framework

### A. Evolutionary QAT: Encoding INT8 into the Mutation Operator

OmniTrain trains SparseCfC policies end-to-end via reinforcement learning, where the reward is a scalar function of collision and forward progress in the F1TENTH simulator and is not a differentiable function of the network's weights. Gradient-based QAT relies on the Straight-Through Estimator

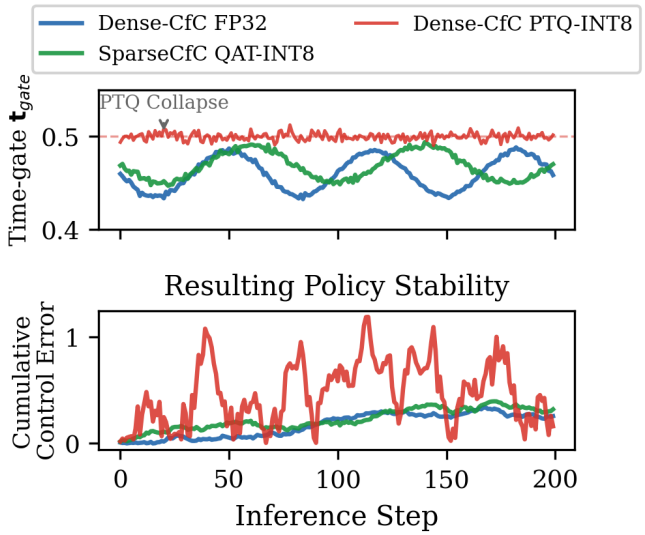


Fig. 1. Time-gate $t_{gate}$ dynamics during inference (top) and resulting cumulative control error (bottom). PTQ immediately collapses the gate to a constant 0.5, rendering the CfC stateless. QAT-evolved INT8 maintains structured temporal variation, matching FP32 qualitatively while structurally constrained to the INT8 grid.

**Algorithm 1** QAT-ES for SparseCfC Training

**Require:** Initial $\theta_0 \in \mathbb{R}^d$, noise $\sigma$, scale $\Delta_q$, generations $G$

**for** $g = 1, \dots, G$ **do**
  Sample $\epsilon_k \sim \mathcal{N}(0, I)$ for $k = 1, \dots, N_{pop}$
  $\tilde{\theta}^{(k)} \leftarrow \theta_g + \sigma\epsilon_k$ ▷ Gaussian perturbation
  $\theta^{(k)}_{QAT} \leftarrow \text{Clamp}(\lfloor \tilde{\theta}^{(k)}/\Delta_q \rceil \times \Delta_q,\ -127,\ 127)$
  Evaluate $F_k \leftarrow \text{Fitness}(\theta^{(k)}_{QAT})$ in F1TENTH simulator
  $\theta_{g+1} \leftarrow \theta_g + \frac{\alpha}{\sigma N_{pop}} \sum_k F_k \epsilon_k$ ▷ ES update
**end for**
**return** $\theta^*_{QAT} \leftarrow$ best $\theta^{(k)}_{QAT}$ over all $G$

(STE) to back-propagate a surrogate gradient through the non-differentiable rounding operator; with no gradient available to begin with, STE has nothing to estimate through. We instead adopt a distributed Evolution Strategy (ES) [9], a black-box optimizer that treats the reward as a fitness signal rather than a loss, making it natively compatible with non-differentiable objectives and allowing the INT8 rounding operator to be embedded directly inside the search: every candidate $\theta^{(k)}$ is snapped to the INT8 grid *before* fitness evaluation,

$$\theta^{(k)}_{QAT} = \text{Clamp}\left( \left\lfloor \frac{\theta + \sigma\mathcal{N}(0,I)}{\Delta_q} \right\rceil \times \Delta_q,\ -127,\ 127 \right) \tag{2}$$

so that fitness is always measured on the network the ESP32-S3 will actually execute, never a full-precision proxy. Over $G = 1{,}000$ generations this applies direct selection pressure toward the weight magnitudes $|f_\theta| > \Delta_q/\Delta t$ that negate Proposition 1's collapse condition, without computing a single derivative. Algorithm 1 formalizes the procedure.

### B. Simulated Quantization: A Numerical Parity Contract

Eq. (2) constrains every weight to the INT8 grid, but the CSR kernel of Section III-D currently evaluates those grid-constrained values with FP32 arithmetic rather than native

TABLE I
MEMORY LAYOUT OF THE `.omnibit` ARCH-4 (SPARSECFC) PAYLOAD

| Segment | Size | Content |
|---|---|---|
| Magic + Arch Flag | 6 B | "OMNI" + ver + 0x04 (CSR) |
| Alignment Pad | 2 B | Reserved |
| Dimensions | 24 B | $d_{in}, d_{model}, d_{out}, N_w, N_t$ |
| Table of Contents | $N_t \times 4$ B | Byte-offsets per CSR tensor |
| CSR Blob | $N_{nz} \times 4$ B | val, col, row, gate heads |

SparseCfC total: 14.2 KB Flash. SRAM weight allocation: 0 bytes.

integer instructions. This is a deliberate scoping decision, not an oversight: it isolates the question this letter answers – whether a CfC's structure survives 8-bit discretization at all – from the orthogonal problem of integer-kernel design, following the same fake-quantization logic used elsewhere in the QAT literature to characterize accuracy before committing to an integer runtime [**?**], [8]. We hold the two representations to a strict numerical contract rather than an approximate one: Section IV-C reports a maximum absolute deviation $|\delta|_{max} < 10^{-4}$ between the on-device FP32 evaluation and an offline PyTorch reference over 1,000-step rollouts, confirming that the reported gains trace to the INT8-constrained weight distribution and not to residual floating-point slack. Native INT8 arithmetic over the same CSR layout is explicit future work (Section V).

### C. SparseCfC Architecture

Rather than pruning a dense network post hoc, we impose the wiring prior of Neural Circuit Policies [5]: sensory, inter-, command-, and motor-neuron populations connected through directed synapses under fixed fan-in/fan-out bounds, in place of full connectivity. This prior is applied to the backbone projection that feeds the candidate branch $g_\theta$, historical branch $h_\theta$, and time-gate $f_\theta$ of (1), through a binary NCP mask $\mathbf{M}$:

$$\mathbf{b}(t) = \tanh((\mathbf{W}_{bb} \odot \mathbf{M})[\mathbf{x}(t); \mathbf{h}(t)] + \boldsymbol{\beta}_{bb}) \quad (3)$$

The fully-connected Dense baseline for F1TENTH contains ∼4,000 parameters (>16 KB in FP32). The Pareto-optimal connectome selected by QAT-ES wires 40 neurons in a 20-10-10 sensory–inter–command population through 270 non-zero backbone synapses, enforcing better than 75% sparsity on the backbone. Table I shows that this sparsity, propagated through the CSR encoding of Section III-D, is what lets the complete model – non-zero weights, dense sensory-encoder and motor-decoder head layers, and CSR index arrays – compile to exactly 14.2 KB. Restricting the sensory layer to $N_{sensory} = 20 < I = 25$ further forces the encoder to compress raw LiDAR geometry into a compact latent representation rather than pass it through unchanged, reducing MSE by 39.5% against a one-to-one mapping.

### D. The `.omnibit` Zero-Copy Format and CSR Engine

The `.omnibit` format (Table I) packs each SparseCfC layer as a CSR blob – values, column indices, row pointers, and gate-head offsets – behind a fixed header of dimensions and per-tensor byte offsets. At load time, OmniEngine issues a single `mmap()` call against the SPI Flash region backing the file, mapping its byte range directly into the Xtensa LX7's address space instead of copying it into a heap buffer. The $O(N_{nz})$ CSR multiply-accumulate kernel runs from IRAM (`IRAM_ATTR`), compiled with `#pragma GCC optimize("O3,unroll-loops")` and `__restrict`-qualified pointers to remove aliasing checks from the hot loop. Because the mapped region is read through the execute-in-place (XIP) path, the ESP32-S3's hardware prefetcher speculatively streams sequential Flash addresses into the instruction cache ahead of the CSR traversal, masking Flash latency behind computation instead of paying it as an up-front block copy (Fig. 2). Weights therefore never occupy a byte of SRAM: the 1.5 KB footprint of Table III is entirely the hidden-state buffer $h(t)$.

TABLE II
TIME-TO-FAILURE UNDER SENSOR PACKET LOSS (STEPS, 30 SEEDS)

| Arch. | 0% Loss | 20% Loss | 60% Loss |
|---|---|---|---|
| LSTM | 500 | 500 | $50.0 \pm 22.6$ |
| GRU | 500 | 500 | $108.8 \pm 136.3$ |
| CfC | 500 | 500 | $257.3 \pm 202.7$ |

CfC vs. LSTM: $p < 0.001$, $d = -1.41$; CfC vs. GRU: $p = 0.002$, $d = -0.85$.

## IV. EXPERIMENTAL EVALUATION

### A. F1TENTH Autonomous Racing: An Ablation Study

**Setup:** 1,000 QAT-ES generations; NVIDIA RTX 5070 for parallel fitness evaluation; F1TENTH Gym [1]; $I = 25$ downsampled LiDAR rays; steering and velocity outputs.

Dense-CfC/FP32 already exceeds the MLP/FP32 baseline (trained via PPO-clip [10]) through continuous-time structure alone; naive PTQ collapses that gain by 82%, confirming Proposition 1 at operating conditions (Fig. 3). QAT-ES not only recovers the loss but inverts it: the resulting SparseCfC reaches a fitness of 52,312, clears the "Wall of Death" obstacle at 318 m, and sustains over 19,000 steps (4.8 km) at a range where accumulated hidden-state divergence fails every FP32 gradient-trained model.

### B. Temporal Jitter Resilience (PiL)

**Setup:** F1TENTH @ 20 Hz; 5,000-step horizon; zero-order-hold (ZOH) LiDAR packet loss; 30 seeds; Welch $t$-test with Holm–Bonferroni correction.

CfC outlasts LSTM by 5.1× and GRU by 2.4× at 60% loss (Table II). The continuous-time ODE extrapolates hidden state over ZOH-held inputs using the true elapsed time, while LSTM's fixed gating accumulates error irreversibly, proportional to loss duration.

### C. Bare-Metal Hardware Benchmarks (ESP32-S3)

OmniEngine improves on every resource dimension in Table III. At 1.22 ms per inference, the ESP32-S3 completes the neural computation in 2.4% of the 50 ms control window, leaving 97.6% of CPU time for IMU fusion and telemetry. Numerical parity with PyTorch was verified over 1,000-step rollouts: maximum absolute deviation $|\delta|_{max} < 10^{-4}$, confirming the FP32 CSR engine evaluates the INT8-constrained weights exactly, per the simulated-quantization contract of Section III-B.

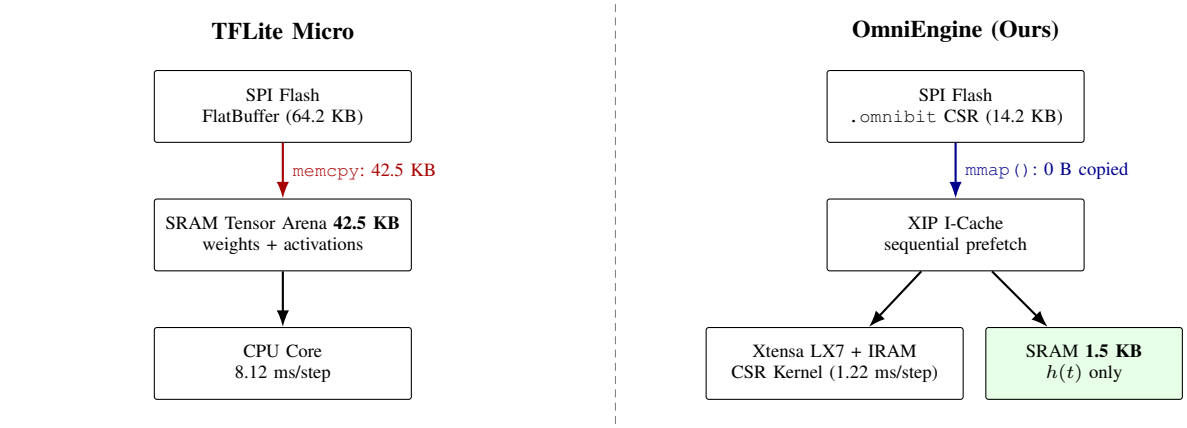


Fig. 2. Memory model comparison. Left (TFLite Micro): the full 42.5 KB weight/activation blob is `memcpy`'d to SRAM before inference. Right (OmniEngine): a single `mmap()` call streams CSR entries directly from SPI Flash to the ALU via the XIP cache, reserving SRAM exclusively for the 1.5 KB hidden-state buffer $h(t)$.

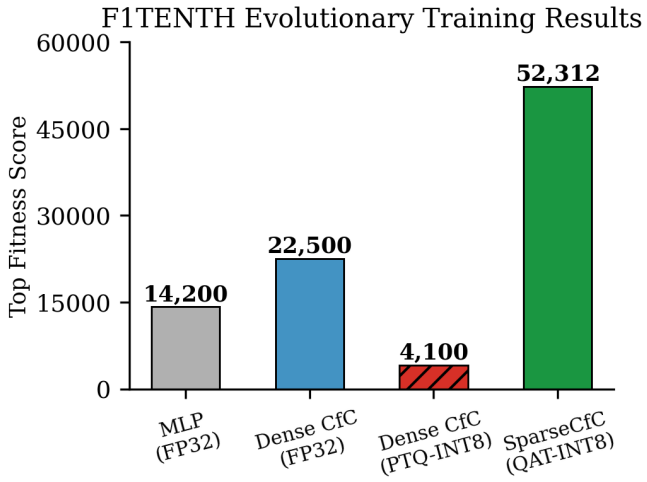


Fig. 3. F1TENTH top fitness across all training configurations. PTQ collapses the Dense CfC by 82%; QAT-ES enables the INT8 SparseCfC to surpass its FP32 predecessor by 132% (>4.8 km continuous navigation). The dashed line marks the Dense-FP32 ceiling.

TABLE III
BARE-METAL UTILIZATION: OMNIENGINE VS. TFLITE MICRO (ESP32-S3 @ 240 MHz)

| Framework | SRAM | Flash | Latency |
|---|---|---|---|
| TFLM / LSTM | 42.5 KB | 64.2 KB | 8.12 ms |
| TFLM / GRU | 38.2 KB | 58.1 KB | 7.45 ms |
| **OmniEngine** | **1.5 KB** | **14.2 KB** | **1.22 ms** |
| Reduction | 96% | 78% | 85% |

### *D. Hardware-in-the-Loop (HIL) Validation*

Injecting pre-recorded LiDAR telemetry directly into the ESP32-S3 over serial confirmed both theoretical predictions on physical hardware: measured inference latency matched the 1.22 ms expectation exactly, with the same $|\delta|_{max} < 10^{-4}$ numerical parity reported in Section IV-C.

## V. CONCLUSION AND FUTURE WORK

OmniTrain resolves the two compounding barriers to bare-metal continuous-time neural network deployment: the SRAM wall imposed by dynamic tensor arenas, and the incompatibility of standard PTQ with CfC time-gates formalized in Proposition 1. QAT-ES produces an INT8-constrained SparseCfC that outperforms its FP32 counterpart by 132%, while the `.omnibit` Zero-Copy format and $O(N_{nz})$ CSR engine deliver a 96% SRAM reduction against TFLite Micro at 1.22 ms per inference – 2.4% of the 20 Hz control budget – on a $5 commodity MCU.

**Future Work:** replacing the simulated-quantization deployment of Section III-B with native INT8 storage and integer arithmetic will shrink the 14.2 KB Flash footprint by a further 4×; Xtensa SIMD vectorization of the CSR kernel targets sub-millisecond inference.

## ACKNOWLEDGMENT

The author thanks the Hasani et al. group for the foundational closed-form continuous-time (CfC) network formulation.

## REFERENCES

[1] M. O'Kelly, H. Zheng, D. Karthik, and R. Mangharam, "F1TENTH: An open-source evaluation environment for continuous control and reinforcement learning," *Proc. Conf. Robot Learning (CoRL)*, 2020.
[2] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
[3] R. Hasani, M. Lechner, A. Amini, D. Rus, and R. Grosu, "Liquid time-constant networks," *Proc. AAAI*, vol. 35, no. 9, pp. 7657–7666, 2021.
[4] R. Hasani, M. Lechner, A. Amini, L. Liebenwein, A. Ray, M. Tschaikowski, G. Teschl, and D. Rus, "Closed-form continuous-time neural networks," *Nature Machine Intelligence*, vol. 4, no. 11, pp. 992–1003, 2022.
[5] M. Lechner, R. Hasani, A. Amini, T. A. Henzinger, D. Rus, and R. Grosu, "Neural circuit policies enabling auditable autonomy," *Nature Machine Intelligence*, vol. 2, pp. 642–652, 2020.
[6] R. David et al., "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," *Proc. Machine Learning and Systems (MLSys)*, 2021.
[7] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, "Neural ordinary differential equations," *NeurIPS*, pp. 6571–6583, 2018.
[8] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, "Quantized neural networks," *J. Machine Learning Research*, vol. 18, pp. 6869–6898, 2017.
[9] T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever, "Evolution strategies as a scalable alternative to reinforcement learning," *arXiv:1703.03864*, 2017.
[10] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv:1707.06347*, 2017.